

Elev8ai MVP Build Plan

Sep 24, 2026 · @Sid

Overview

The MVP proves one loop: young people sign up, take paid jobs from your own companies, buy a course to unlock better jobs, and get paid. Target: a working product in about 8 weeks, built with Claude Code by you or one developer.

```
flowchart LR
    A[Sign up] --> B[Take open job]
    B --> C[Get paid]
    A --> D[Buy course]
    D --> E[Submit assignment]
    E --> F[Earn badge]
    F --> G[Unlock gated jobs]
    G --> C
```

The earning path and the learning path both end in paid work, which is the whole value of Elev8ai.

Launch companies (posted by admin): your agency (cold calling, content, clipping), your podcast (clipping), your clothing brand (content creation), and the web dev agency (small badge-gated dev tasks).

In the MVP:

- Talent signup, login and profile.
- Admin panel: create companies, post jobs, review applications, approve submissions, grade assignments, mark payouts.
- Job board with filters (category, pay, open vs badge-required).
- Job flow: apply, accept, submit proof, approve or request changes, get paid.
- Chat thread per job between talent and the company (you or your staff).
- In-app and email notifications.
- Course store: pay once, watch lessons, submit one assignment, get graded, earn a badge.
- Earnings page and payout onboarding.

Not in the MVP: self-serve company accounts, escrow, subscription tiers, agency seats, AI grading, ratings, mobile app, push notifications.

MVP revenue: course sales only. Jobs are funded by your own companies, so they are a cost of validating the model, not a revenue line yet.

Accounts and services to create

You need about 14 accounts before or during the build; most are free until launch, and running costs at launch should be roughly \$60–120 per month plus payment fees. All prices below are approximate, so check each provider's pricing page when you sign up.

Payments warning for Mauritius: Stripe does not onboard Mauritius-registered businesses as platforms, and the usual workaround is forming a US LLC ([source](#)). For the MVP, avoid that complexity: sell courses through a merchant-of-record like Lemon Squeezy, and pay talent manually. Move to Stripe Connect in V2 once you have a supported entity.

#	Account	What it is for	Create by	Approx. cost
1	Domain (Cloudflare Registrar or Namecheap)	elev8ai domain for the app and email	Week 0	\$10–15/year
2	Business email (Google Workspace or Zoho Mail)	hello@, support@, payouts@ addresses	Week 0	Free (Zoho) to ~\$7/user/month
3	Claude subscription (Pro or Max)	Runs Claude Code, which writes the app	Week 0	See claude.com/pricing
4	GitHub	Stores the code, version history	Week 0	Free
5	Supabase	Database, login, file storage, realtime chat	Week 0	Free in dev, ~\$25/month at launch
6	Vercel	Hosts the Next.js app	Week 0	Free in dev, ~\$20/month at launch (Hobby plan is non-commercial)
7	Resend	Sends notification and login emails from your domain	Week 1	Free tier covers the MVP

#	Account	What it is for	Create by	Approx. cost
8	Lemon Squeezy (or MIPS / Paywise if buyers are mostly in Mauritius)	Takes course payments, handles sales tax	Week 5	~5% + \$0.50 per sale
9	Wise Business and/or PayPal Business	Paying international talent	Week 6	Per-transfer fees
10	Local bank / MCB Juice business	Paying talent in Mauritius	Week 6	Bank fees
11	Bunny Stream (or Mux)	Private course video hosting	Week 5	~\$1-10/month at MVP scale
12	Cal.com or Calendly	Booking links for cold-calling jobs, proof that meetings happened	Week 3	Free
13	Sentry	Error alerts when something breaks	Week 7	Free
14	PostHog	Product analytics and funnels	Week 7	Free tier

Non-technical setup:

- ☐ Register the business (Corporate and Business Registration Department in Mauritius) if not done through one of your existing companies.
- ☐ Check data-protection registration duties with the Mauritius Data Protection Office, since you store personal data.
- ☐ Terms of Service, Privacy Policy and Independent Contractor Agreement (a template tool like Termly to start, then a lawyer review).
- ☐ Instagram, TikTok and LinkedIn handles for Elev8ai to recruit talent.
- ☐ A password manager (Bitwarden or 1Password) shared with anyone helping you; turn on two-factor login on every account above.

Tech stack and architecture

One Next.js app on Vercel talks to Supabase for everything stateful; this is the stack Claude Code writes most reliably and one person can run.

Layer	Choice	Why
Frontend + backend	Next.js (App Router) + TypeScript	One codebase for pages and server logic
Styling	Tailwind CSS + shadcn/ui	Fast, consistent UI components
Database	Supabase Postgres	Relational data fits jobs, applications, badges
Auth	Supabase Auth (email magic link + Google)	No passwords to manage
Security	Row Level Security (RLS) policies	Talent only ever sees their own data
Files	Supabase Storage (private buckets)	Submissions, assignment uploads, avatars
Chat	Supabase Realtime	Live messages without a paid chat service
Email	Resend + React Email	Branded notification emails
Course video	Bunny Stream with signed URLs	Buyers-only playback
Course payments	Lemon Squeezy checkout + webhook	Purchase unlocks the course automatically
Payouts	Manual transfer, logged in admin	No payment-platform dependency in MVP
Validation	Zod	Checks every form and API input
Testing	Playwright	Tests the key flows end to end

```

flowchart LR
    U[Talent browser] --> N[Next.js on Vercel]
    A[Admin browser] --> N
    N --> S[(Supabase DB + Auth)]
    N --> F[Supabase Storage]
    N --> R[Resend email]
    L[Lemon Squeezy] -- webhook --> N
    N --> B[Bunny video]

```

Every request goes through the Next.js app, and only Lemon Squeezy calls in from outside, through a signed webhook.

Two roles in the MVP: `talent` (default on signup) and `admin` (you and your staff, set by hand in the database). Companies are records the admin manages, not user accounts yet.

Database schema

Sixteen tables cover the whole MVP; money is stored in cents as integers, and every table has `id` (uuid) and `created_at`.

Table	Key fields	Notes
profiles	user_id, role (talent/admin), full_name, username, avatar_url, country, date_of_birth, bio, phone	One per auth user; signup blocked under 18
payout_details	user_id, method (bank/juice/wise/paypal), details (json)	Readable only by the owner and admins
companies	name, slug, logo_url, description, website	Admin-managed; your 4 launch companies
skills	slug, name, description	e.g. clipping, cold_calling, content, web_dev
user_skills	user_id, skill_id, source (course/manual), awarded_by	These are the badges
jobs	company_id, title, description, category, pay_cents, pay_type (fixed/per_unit), unit_label, max_units, slots, required_skill_id (nullable), deadline, proof_instructions, status (draft/open/closed)	required_skill_id empty = open to all
applications	job_id, user_id, pitch, status (pending/accepted/rejected/withdrawn)	Unique per job + user
submissions	application_id, notes, links[], file_paths[], units_claimed, status (submitted/changes_requested/approved/rejected), reviewer_note	Proof of work
payouts	user_id, submission_id, amount_cents, status (owed/paid), method, reference, paid_at	Created on approval, marked paid by admin
conversations	application_id	One chat thread per accepted job
messages	conversation_id, sender_id, body, attachment_path, read_at	Realtime enabled
notifications	user_id, type, title, body, link, read_at, emailed_at	Bell icon + email
courses	slug, title, description, price_cents, skill_id, lemon_variant_id, published	Passing awards skill_id
lessons	course_id, position, title, body_md, video_id	Ordered lessons
course_access	user_id, course_id, order_id, amount_cents	Created by payment webhook

Table	Key fields	Notes
course_assignments	user_id, course_id, notes, links[], file_paths[], status (pending/passed/failed), feedback, graded_by	Manual grading

A job moves through these states for each worker:

```
stateDiagram-v2
    [*] --> Applied
    Applied --> Accepted
    Applied --> Rejected
    Accepted --> Submitted
    Submitted --> ChangesRequested
    ChangesRequested --> Submitted
    Submitted --> Approved
    Submitted --> RejectedWork
    Approved --> PayoutOwed
    PayoutOwed --> Paid
    Paid --> [*]
```

Every arrow sends a notification to the other side, which is how talent always knows where their money is.

Build timeline

Eight weeks from empty repo to public launch, working in seven build phases; each phase ends with something you can click through and test.

Week	Phase	Done when
0	Setup: accounts, Claude Code, repo, first course outlines	Every account in the table above exists; blank app deploys to Vercel
1	1. Foundation: auth, profiles, roles, app layout	You can sign up, finish onboarding, and see an empty dashboard; admin sees an admin area
2	2. Companies and jobs: admin posting, talent job board	Admin posts a clipping job for the podcast; talent sees and filters it
3	3. Job workflow: apply, accept, submit, review, payout ledger	A test user completes a job end to end and a payout shows as owed

Week	Phase	Done when
4	4. Chat and notifications	Admin and talent message live; each status change sends in-app + email alerts
5	5. Courses: catalog, lessons, video, checkout	Paying for Clipping 101 in test mode unlocks its lessons
6	6. Assignments, badges, gated jobs, payout details	Passing an assignment awards a badge that unlocks a cold-calling job
7	7. Launch prep: landing page, legal pages, analytics, security review, tests	Every item in the launch checklist is ticked
8	Private beta (20–30 talent), fixes, public launch	First real payouts sent; first course sales

If you fall behind, cut from week 7 polish, never from the payout flow or the badge loop.

Claude Code setup

Install Claude Code, give it a CLAUDE.md that describes Elev8ai, then feed it the phase prompts one at a time; setup takes about an hour. Claude Code needs a Claude subscription or an Anthropic Console account ([docs](#)).

1. Install the tools on your computer

- Node.js (current LTS), Git, and VS Code.
- Claude Code, using the native installer from the official docs:

```
# macOS, Linux, WSL
curl -fsSL https://claude.ai/install.sh | bash

# Windows PowerShell
irm https://claude.ai/install.ps1 | iex
```

Prefer not to use a terminal? The Claude Desktop app also runs Claude Code.

2. Create the project

```
mkdir elev8ai && cd elev8ai
git init
claude
```

You log in on first launch. Then create the CLAUDE.md below in the project root before running any phase prompt.

3. CLAUDE.md (paste as-is, edit as the project evolves)

```
# Elev8ai
```

Marketplace where young people (18+) take paid micro-jobs from companies and buy courses that unlock skill-gated jobs.

```
## MVP scope
```

- Roles: talent (default) and admin. Companies are admin-managed records.
- Job categories: clipping, cold_calling, content, web_dev.
- Flow: apply -> accept -> submit proof -> approve -> payout owed -> paid.
- Courses: pay once (Lemon Squeezy), lessons, one assignment, manual grading, pass awards a skill badge that unlocks gated jobs.
- Chat per accepted application. In-app + email notifications.
- Payouts are manual transfers logged by admin. No escrow, no subscriptions.

```
## Stack
```

Next.js App Router, TypeScript strict, Tailwind, shadcn/ui, Supabase (Postgres, Auth, Storage, Realtime), Resend, Zod, Playwright.

```
## Rules
```

- Money is integer cents. Never floats.
- Every table has RLS enabled. Talent can only read/write their own rows. Admin checks go through a SQL function `is_admin()`.
- All DB changes are SQL migration files in `supabase/migrations`.
- Validate every form and server action input with Zod.
- Server-only secrets never reach client components.
- Mobile-first UI; most talent will use phones.
- Small, reviewable changes. Explain the plan before large edits.
- After each feature: run typecheck, lint, and relevant tests.

```
## Commands
```

```
npm run dev | npm run build | npm run lint | npm run typecheck | npm run test:e2e
```

4. How to work with it

- One prompt from the next section at a time. Each starts by asking for a plan; read it, correct it, then let it build.
- Commit to GitHub after every working step, so you can always roll back.
- Start a fresh Claude Code session at the start of each phase so it works from CLAUDE.md and the code, not a long old conversation.
- Put secrets only in `.env.local` (never committed) and in Vercel's environment settings. Never paste keys into prompts.

- Test every flow yourself on your phone before moving on. When something breaks, paste the exact error and what you did.

Claude Code prompts

Run these in order, one per session step, and commit after each works. Each prompt assumes CLAUDE.md is in place. Before Phase 2, copy the Database schema table from this doc into docs/schema.md in the repo, because several prompts point Claude Code to it.

Phase 0: Scaffold

```
Read CLAUDE.md. Show me a plan first, then scaffold the project:
- Next.js App Router, TypeScript strict, Tailwind, shadcn/ui, ESLint, Prettier.
- Scripts: dev, build, lint, typecheck, test:e2e (Playwright installed).
- Supabase: init the supabase/ folder for migrations, add @supabase/ssr helpers for browser client, server client, and middleware session refresh.
- .env.example listing every variable we will need (Supabase URL, anon key, service role key, Resend key, Lemon Squeezy webhook secret, Bunny keys, app URL) with a comment on each. Make sure .env.local is gitignored.
- A placeholder home page that says Elev8ai.
Run build and typecheck and fix anything that fails.
```

Phase 1: Foundation

1a. Profiles and roles

```
Plan first. Create a migration for:
- profiles (user_id pk references auth.users, role text check in ('talent','admin') default 'talent', full_name, username unique, avatar_url, country, date_of_birth, phone, bio, onboarded boolean).
- A trigger that inserts a profiles row when a new auth user is created.
- A SQL function is_admin() returning true if the current user's role is admin.
- RLS: users read and update only their own profile and cannot change their own role; admins read all profiles.
- A public avatars storage bucket with policies so users can only write to their own folder.
Generate TypeScript types from the schema and explain how I apply the migration to my Supabase project.
```

1b. Auth and onboarding

Plan first. Build authentication:

- /login with email magic link and Google sign-in via Supabase Auth.
- Auth callback route and middleware that protects /app/** and /admin/**.
- /onboarding form (full name, username, date of birth, country, phone, short bio, avatar upload). Reject anyone under 18 with a friendly message. Validate with Zod in a server action.
- After login: not onboarded -> /onboarding; admin -> /admin; else /app/jobs.
- Logout button.

1c. App shell

Plan first. Build the layouts:

- Talent area (/app): mobile-first, bottom navigation on phones and a sidebar on desktop with Jobs, My Jobs, Learn, Messages, Profile, plus a notification bell in the header (placeholder for now).
- Admin area (/admin): sidebar with Companies, Jobs, Applications, Submissions, Payouts, Courses, Grading, Users. Only admins can open it, checked on the server.
- Clean, modern style: dark navy and electric-green accent, rounded cards. Add empty placeholder pages for each nav item.

Phase 2: Companies and jobs

2a. Schema

Plan first. Migration for companies, skills, user_skills and jobs exactly as described in docs/schema.md (jobs: company_id, title, description, category in clipping/cold_calling/content/web_dev, pay_cents int, pay_type in fixed/per_unit, unit_label, max_units, slots, required_skill_id nullable, deadline, proof_instructions, status in draft/open/closed).

RLS: anyone logged in can read companies, skills, and jobs with status open; only admins can write. user_skills: users read their own, admins manage all. Seed 4 skills (clipping, cold_calling, content, web_dev) and 4 companies (my agency, my podcast, my clothing brand, the web dev agency) with placeholder descriptions I can edit.

2b. Admin posting

Plan first. Build admin pages:

- Companies: list, create, edit, logo upload.
- Jobs: list with filters by company/status/category; create and edit form with every job field; required skill picker ('Open to everyone' or a skill); save as draft or publish; close a job; duplicate a job.
- Show pay nicely (e.g. \$5 per clip, max 20 clips).

2c. Talent job board

Plan first. Build /app/jobs for talent:

- Cards with company logo, title, pay, category, deadline, spots left.
- Filters: category, minimum pay, 'jobs I can take now' vs 'locked'.
- Locked jobs (required skill the user lacks) show a lock and a link to the course that awards that skill (leave the link to /app/learn for now).
- Job detail page with full description and proof instructions.
- Empty states and loading skeletons. Works well on a phone.

Phase 3: Job workflow

3a. Schema

Plan first. Migration for applications, submissions and payouts (fields as in docs/schema.md: applications unique per job+user; submissions with links[], file_paths[], units_claimed, status, reviewer_note; payouts with amount_cents, status owed/paid, method, reference, paid_at). Private 'submissions' storage bucket: users write only to their own folder, admins read all. RLS: talent see only their own applications, submissions and payouts; admins see all.

3b. Apply and accept

Plan first. Build:

- Apply button with optional short pitch on the job detail page.
On the server, block: already applied, job not open, no slots left, missing required skill. Show clear reasons.
- /app/my-jobs with tabs: Applied, In progress, Submitted, Completed.
- /admin/applications queue: accept or reject with optional note.
Accepting counts against slots; auto-close the job when slots are full.

3c. Submit, review, payout ledger

Plan first. Build:

- Submission form on an accepted job: links, file uploads, units claimed (for per_unit jobs), notes. Show the job's proof instructions above it.
- /admin/submissions review queue with approve, request changes (note required), reject (note required).
- Approval runs in one Postgres function (transaction): mark approved and create a payout row owed = pay_cents for fixed jobs, or pay_cents x min(units approved, max_units) for per_unit jobs. Admin can adjust approved units.
- /app/earnings: owed, paid, lifetime total, and history.
- /admin/payouts: list owed payouts grouped by user; mark paid with method and reference; CSV export.

Add a Playwright test for apply -> accept -> submit -> approve -> paid.

Phase 4: Chat and notifications

4a. Chat

Plan first. Build per-job chat with Supabase Realtime:

- Migration: conversations (one per accepted application, created automatically on acceptance) and messages (sender_id, body, attachment_path, read_at). RLS: only the application's talent and admins can read or write. Enable realtime on messages.
- /app/messages inbox and /admin/messages inbox: threads sorted by latest message, unread counts, job title shown on each thread.
- Thread view: live updates, image/file attachments (private bucket), mark as read on open, 'Open job' link.
- Keep people from sharing phone numbers or emails in chat for now by showing a gentle warning if a message contains one (do not block).

4b. Notifications

Plan first. Build notifications:

- Migration for notifications (as in docs/schema.md) plus an email_opt_out boolean on profiles. RLS: users see only their own.
- A server helper notify(userId, type, title, body, link) that inserts the row and, unless opted out, sends an email through Resend with a React Email template in Elev8ai branding.
- Fire it on: application accepted/rejected, submission approved/changes requested/rejected, payout marked paid, new message (email at most once per thread per hour), new job posted in a category the user has worked in, course purchased, assignment graded. Admins get: new application, new submission, new assignment to grade.
- Header bell with live unread count and a dropdown of the latest 10; /app/notifications full list; mark all as read.

Phase 5: Courses

5a. Course admin

Plan first. Migration for courses, lessons and course_access (see docs/schema.md). RLS: anyone logged in reads published courses and lesson titles; lesson bodies and video ids only for users with course_access or admins. Admin pages: create/edit course (title, description, price, skill awarded, Lemon Squeezy variant id, published), lesson editor with ordering, markdown body with preview, and Bunny video id.

5b. Learning pages

Plan first. Build /app/learn:

- Catalog of published courses with price and the jobs each unlocks.
- Course page: syllabus, price, 'Buy' button, or 'Continue' if owned.
- Lesson page for owners: markdown body and a Bunny Stream player using a signed embed URL generated on the server that expires after a few hours.
- Mark lesson complete, progress bar, next/previous lesson.
- Replace the job-board 'locked' link so it points at the right course.

5c. Payments

Plan first. Integrate Lemon Squeezy:

- 'Buy' opens the Lemon Squeezy checkout for the course's variant, passing the user id as custom data and prefilling their email.
 - Route /api/webhooks/lemonsqueezy verifies the signature with the webhook secret, handles order_created, is idempotent on order id, inserts course_access, and calls notify(). Handle refunds by removing access.
 - A success page that waits for access to appear, then opens lesson 1.
- Explain how to test this in Lemon Squeezy test mode locally.

Phase 6: Badges and gated jobs

6a. Assignments and grading

Plan first. Build course assignments:

- Add an assignment_brief field to courses. After the last lesson, show the brief and a submission form (links, files, notes). One pending submission at a time; resubmission allowed after a fail.
- /admin/grading queue: view submission, pass or fail with required feedback. Pass inserts the course's skill into user_skills in the same transaction. Notify the user either way.
- Show earned badges on the profile and next to the user's name in admin.
- Admin can also award or revoke a badge manually from the Users page.

6b. Unlock moment and payout details

Plan first. Build:

- When a badge is awarded, notify the user with a link to the job board filtered to jobs that badge unlocks; show a 'Newly unlocked' tag on those jobs for 7 days.
- /app/settings/payout: choose method (bank transfer, MCB Juice, Wise, PayPal) and enter details. Stored in payout_details, readable only by the owner and admins. Talent must fill this before their first payout.
- Show each user's payout details on /admin/payouts next to owed amounts.

Phase 7: Launch prep

7a. Public pages

Plan first. Build the public site:

- Landing page: hero ('Get paid for real work. Learn skills that pay more.'), how it works in 3 steps, job categories, featured courses, launch companies, FAQ, sign-up call to action.
- /terms, /privacy and /contractor-agreement pages that render markdown files from /content so I can paste in final legal text.
- SEO metadata, Open Graph image, sitemap, robots.txt, favicon.

7b. Security review

Act as a strict security reviewer. Audit every RLS policy, storage policy, server action, API route and webhook. Try to find ways a talent user could read another user's data, approve their own submission, change their role, grant themselves course access or a badge, or see payout details. List findings by severity with file paths, then fix them one by one and add tests that prove each fix.

7c. Tests, monitoring, production

Plan first. Then:

- Playwright tests for: signup + onboarding, under-18 rejection, job apply-to-paid flow, gated job blocked then unlocked after passing an assignment, course purchase via a mocked webhook, chat message delivery.
- Add Sentry error tracking and PostHog analytics with events: signed_up, onboarded, job_applied, job_accepted, submission_approved, payout_paid, course_viewed, course_purchased, assignment_passed.
- Write docs/deploy.md: every production environment variable, creating a separate production Supabase project, applying migrations, verifying the Resend domain, switching Lemon Squeezy to live mode and the live webhook.

Bug-fix template

Something is broken. What I did: <steps>. What I expected: <expected>. What happened: <actual>. Exact error: <paste>. Find the root cause before changing code, explain it in two sentences, fix it, and add a test so it cannot come back.

Content and operations prep

While the app is being built, prepare the courses, the first jobs and the payout routine in parallel; without them the finished app launches empty.

Courses (record by week 5)

Course	Price	Lessons	Assignment	Pass criteria	Unlocks
Clipping 101	~\$12	5–6 short videos: finding the hook, cutting, captions, formats per platform, posting	3 clips from a podcast episode you provide	Hook in first 3 seconds, clean captions, correct format, on-brand	Paid podcast and agency clipping jobs
Cold Calling & Appointment Setting	~\$39	6–8 videos: prospect research, openers, objection handling, booking, CRM notes	Recorded 3–5 minute mock call plus a call script	Rubric score of 7/10 or more	Cold-calling jobs
Content Creation (optional at launch)	~\$19	5 videos: shooting product content, hooks, editing, brand guidelines	2 short videos for the clothing brand	Usable as-is by the brand	Advanced content jobs

First jobs (ready by week 7)

- ☐ Write 10–15 job posts across the four companies, with exact proof instructions for each.
- ☐ Open starter jobs (no badge): simple content ideas, basic clips at \$1–5 each, so free users earn quickly.
- ☐ Gated jobs: podcast clips at a higher rate, cold-calling at a set price per meeting held, one small web dev task.
- ☐ Decide each company's monthly job budget so payouts never surprise you.

Operating rules

Process	Rule
Review submissions	Within 48 hours
Grade assignments	Within 72 hours, with written feedback
Payouts	Weekly, every Friday, for everything approved by Thursday
Minimum payout	\$10, to keep transfer fees sensible
Cold-calling proof	Meeting counts only once it actually takes place
Clipping proof	Posted link plus a views screenshot after 48-72 hours
Support	support@ email answered within 1 business day

Beta recruitment (weeks 6-8)

- ☐ Recruit 20-30 beta users from your networks, your podcast audience and local universities.
- ☐ Give beta users Clipping 101 free in exchange for feedback.
- ☐ Final legal text for Terms, Privacy Policy and Independent Contractor Agreement, reviewed by a lawyer.

Testing and launch checklist

Launch only when every box below is ticked; the security and money items are non-negotiable.

Manual test run (on a phone and a laptop)

- ☐ New user signs up, is rejected if under 18, completes onboarding.
- ☐ Talent applies to an open job, is accepted, chats with admin, submits proof, gets approved, sees payout owed, then paid.
- ☐ Gated job cannot be applied to without the badge, even by editing the URL.
- ☐ Talent buys a course in test mode, watches lessons, submits the assignment, passes, and can now apply to the gated job.
- ☐ Every status change produced an in-app notification and an email.
- ☐ A second test user cannot see the first user's applications, submissions, messages, payouts or payout details.

Security and money

- ☐ Security review prompt (7b) run and every finding fixed.
- ☐ RLS enabled on every table (check in the Supabase dashboard).
- ☐ Service role key used only on the server; not in any client bundle.
- ☐ Webhook signature verified; replaying the same order does not grant access twice.
- ☐ Supabase daily backups on for the production project.

Production setup

- ☐ Separate production Supabase project with all migrations applied.
- ☐ Production environment variables set in Vercel; custom domain connected with HTTPS.
- ☐ Resend domain verified (SPF/DKIM) so emails don't land in spam.
- ☐ Lemon Squeezy in live mode with the live webhook; one real \$1 test purchase refunded.
- ☐ Sentry and PostHog receiving events from production.
- ☐ Final legal pages published; admin roles set for you and your staff.

Launch day

- ☐ 10–15 jobs open, at least 2 courses published.
- ☐ Announce on your podcast, agency and brand channels.
- ☐ Check error alerts and the review queues twice a day for the first two weeks.

Metrics to track after launch

Review these weekly for the first 8 weeks; they decide whether to build V2 (subscriptions and outside companies) and where to set tier limits. Targets are starting guesses to revise with real data.

Metric	Starting target	What a miss tells you
Signup to onboarded	70%	Onboarding is too long
Onboarded to first application within 7 days	50%	Not enough open starter jobs
Accepted jobs completed and approved	70%	Jobs unclear or proof too hard
Median submission review time	Under 48 hours	Your team is the bottleneck
Course page view to purchase	5–10%	Price or the unlock promise is weak

Metric	Starting target	What a miss tells you
Buyers who submit the assignment	60%	Course too long or brief unclear
Assignment pass rate	50–70%	Too low: course weak; too high: bar too easy
Badge holders who take a gated job within 14 days	60%	Not enough gated jobs to reward the purchase
Talent active again after 30 days	30%	Not enough ongoing work
Course revenue vs job payouts	Track the ratio	How much the marketplace costs you to run

The single most important number is the badge-to-gated-job rate: if people who pay to learn quickly earn more, the whole Elev8ai model works.